

Transport Layer

Now we are going onto our next layer: Transport layer.

Its services & protocols:

It provides **logical communication** between application processes running on different hosts.

Transport protocols actions in end systems
e.g PCs:

- Sender: **break** app messages into **segment**. & pass to network layer

- receiver: **reassemble** segments to **messages**. & pass to app layer.

PDU: segment.

2 transport layer ^{protocols} available.

- 1) TCP
- 2) UDP

Understanding difference in ~~same~~ transport layer & network layer protocols.

Network layer: logical communication between **hosts** (~~and~~ devices).

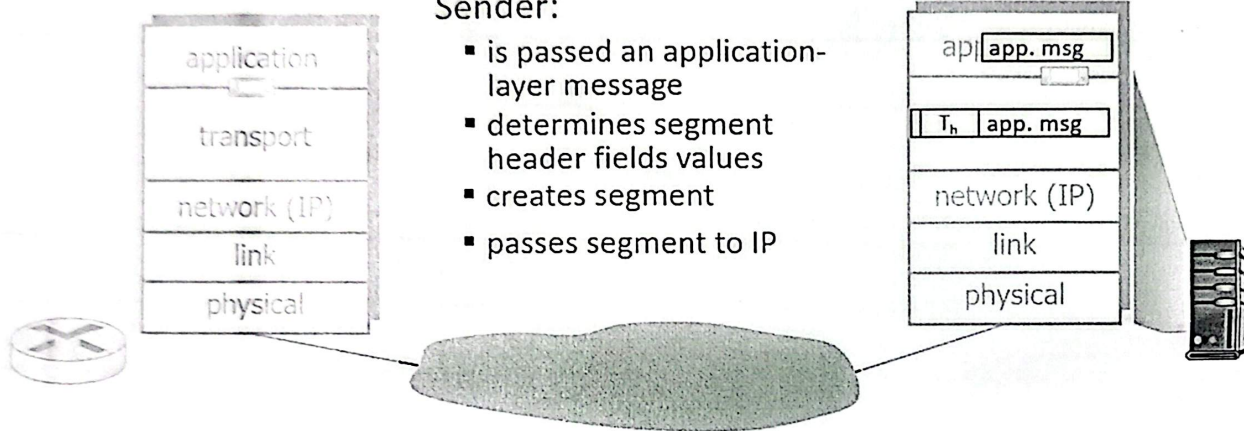
Transport layer: logical communication between **processes**.

remember processes run inside hosts.

Transport layer Actions

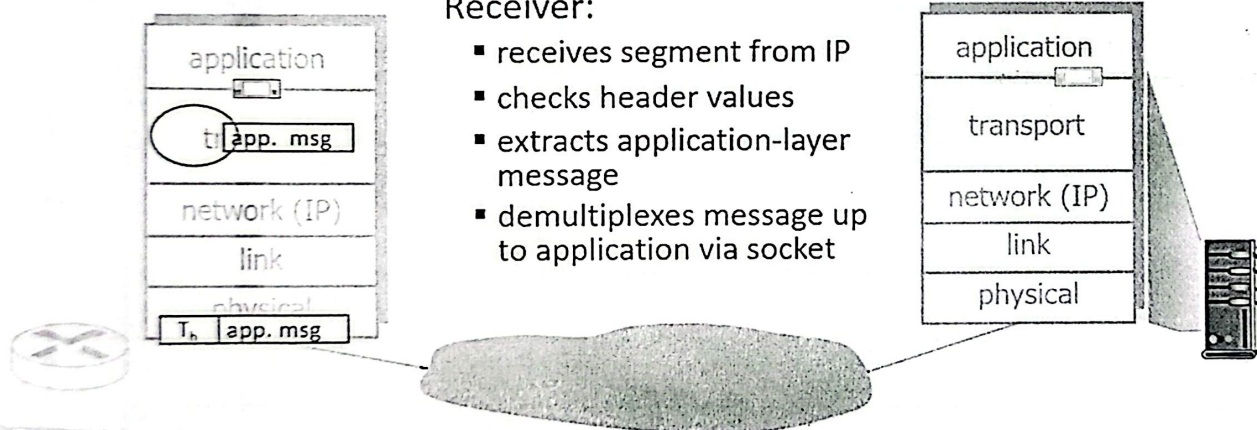
Sender:

- is passed an application-layer message
- determines segment header fields values
- creates segment
- passes segment to IP



Receiver:

- receives segment from IP
- checks header values
- extracts application-layer message
- demultiplexes message up to application via socket



date

2 principal Internet transport protocols

TCP: Transmission Control Protocol

- reliable, in-order delivery
- congestion control
- flow control
- connection setup.

UDP: User datagram protocol:

- unreliable, unordered delivery.
- no built-in extensions of best-effort IP.

Multiplexing & demultiplexing

At transport layer, the job is to deliver data not just to the host device but to the specific application socket running on that host.

Multiplexing: (at sender host): Gathering data chunks from multiple application sockets, encapsulating each chunk with transport header fields (such as source / destination ports) to create segments & passing to network layer

Demultiplexing: (At receiver host): Examine header fields of an incoming transport-layer segment to identify the correct receiving socket & deliver the segment's payload to that exact socket. So remove (h_t) & send the message to correct socket.

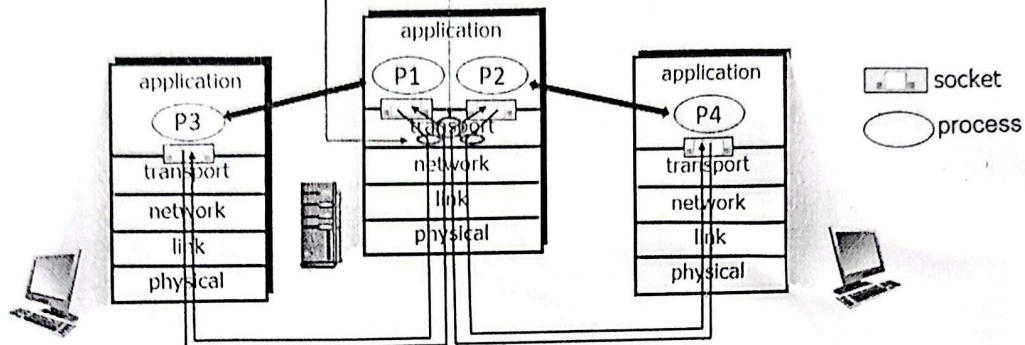
BHATTI

multiplexing at sender:

handle data from multiple sockets, add transport header (later used for demultiplexing)

demultiplexing at receiver:

use header info to deliver received segments to correct socket



Port numbers & socket Addressing.

Every transport layer segment contains 2 vital fields in its header:

- Source Port number (16 bits).

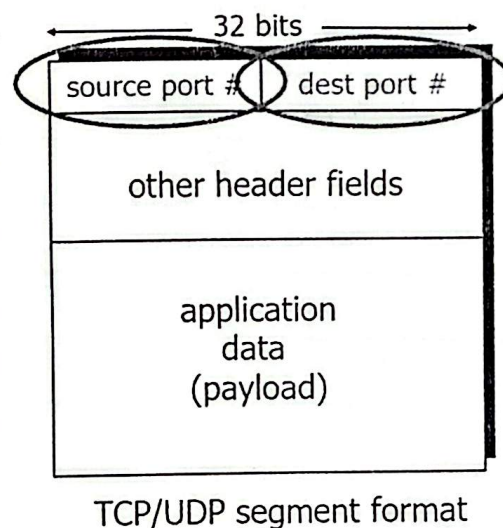
- Destination ports number (16 bits).

- Port numbers range from 0 - 65535 as $2^{16} - 1 = 65535$.

BHATTI

well-known ports : (0-1023) : Reserved for standard protocols like HTTP, DNS, SMTP etc.

Dynamic ports (1024-65535) : Automatically assigned by host OS when a client app opens a socket.



* host gets Port number from transport header, segment's. But the IP address comes from network layer header & its PDU: datagram. Each datagram has a segment which has a Port.

By using IP + port, Host can direct segment to correct socket.

How demultiplexing work?

- host receives IP datagrams
 - each datagram has source IP address, destination IP address
 - each datagram carries one transport-layer segment
 - each segment has source, destination port number
- host uses *IP addresses & port numbers* to direct segment to appropriate socket

* remember datagram are from network layer.
we need IP for identifying host system & port to identify particular process.

There are 2 ways of demultiplexing depending upon UDP or TCP.

- 1) Connectionless Demultiplexing (UDP)
- 2) Connection-oriented demultiplexing (TCP)

1) Connection-less demultiplexing: (UDP):

This is for UDP sockets.

In this, when making socket, host local port number must be specified.

It is identified by a 2-tuple:

UDP socket identifier = (Destination IP address, Destination Port number)

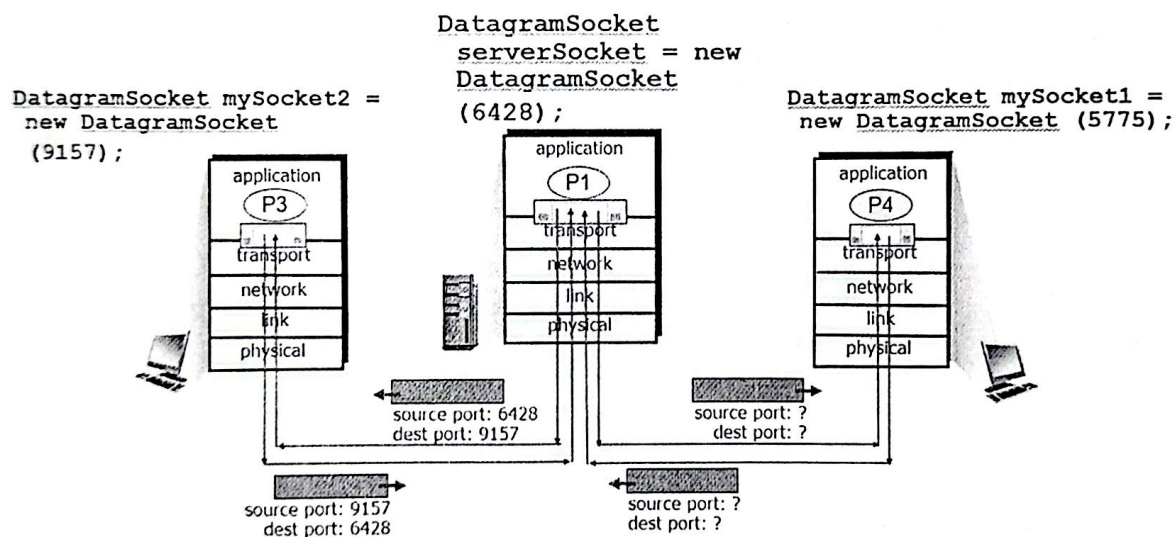
UDP socket identifier should be added by the sender so that it can be sent to the correct device & process.

How it works?

- When Host A creates a UDP segment, it includes a source port & destination port.

• When Host B receives a UDP segment, its **transport layer** checks **only the Destination Port number** & **destination IP** to direct segment to socket.

- **IMP:** If 2 different client hosts send UDP segments to the same destination IP & port e.g DNS server on port 53, **both segments will be directed to same socket** - regardless of **source IP's or ports**.



2) Connection-oriented multiplexing (TCP):

This is for TCP sockets.

A TCP socket is identified by a
4-Tuple:

TCP socket identifier = { Source IP, Source Port, Destination IP, Destination Port }

How it works?

- When a client connects to a TCP server (e.g. a web server on port 80), server accepts connection & creates a new, dedicated socket bound to that 4-tuple.

~~each socket~~

~~IM~~

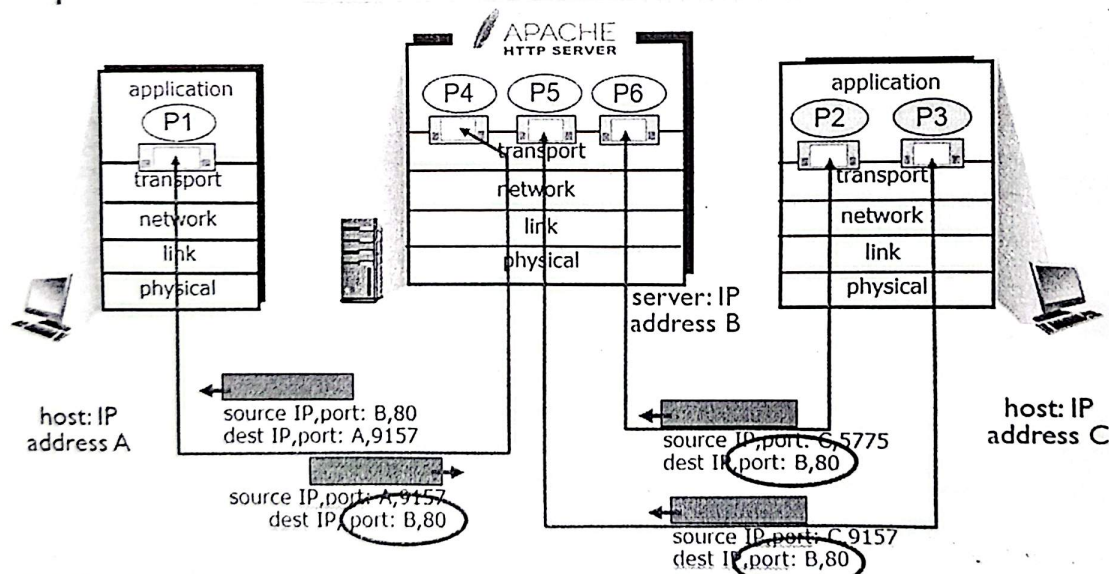
IMP: even if 100 different client browsers send HTTP get requests to Port 80 of same server:

day/date

• each client has a different
(source IP, source port)

• The server uses the full
4-tuple to demultiplex each
packet to its own dedicated
connection socket.

• This allows TCP to maintain
independent state, sequence
numbers, flow control &
timing for every connection.



Three segments, all destined to IP address: B,
dest port: 80 are demultiplexed to different sockets

day/date

UDP vs TCP in connection-Oriented or not.

Feature	UDP (Connectionless)	TCP (Connection-Oriented)
Socket Identifier	2-Tuple (Dest IP, Dest Port)	4-Tuple (Src IP, Src Port, Dest IP, Dest Port)
Server Sockets	Single socket handles requests from all clients	One welcoming socket (listen), plus one dedicated connection socket per active client
Demultiplexing Logic	Checks destination port only	Checks all 4 fields to match exact connection state
Header Overhead	Minimal (8 bytes)	Larger (20 bytes minimum)

BHATTI

UDP : User Datagram protocols:

It is a minimal, lightweight protocol.
It is described as **no-frills** because it almost adds nothing to the underlying network layer (IP).

- It is **connectionless**:

- no handshake needed

- each UDP segment handled independently of others.

- **Unreliable Delivery:**

- Delivery is not guaranteed

- No flow or congestion control.

- **Low overhead:**

A UDP header is only 8 bytes long compared to TCP's 20 bytes header.

UDP is used in:

- streaming
- DNS
- SNMP
- HTTP/3

If reliable transfer needed over UDP:

- add reliability at app layer
- add congestion control at app layer.

e.g. HTTP/3, QUIC.

UDP sender / receiver actions:

Sender actions:

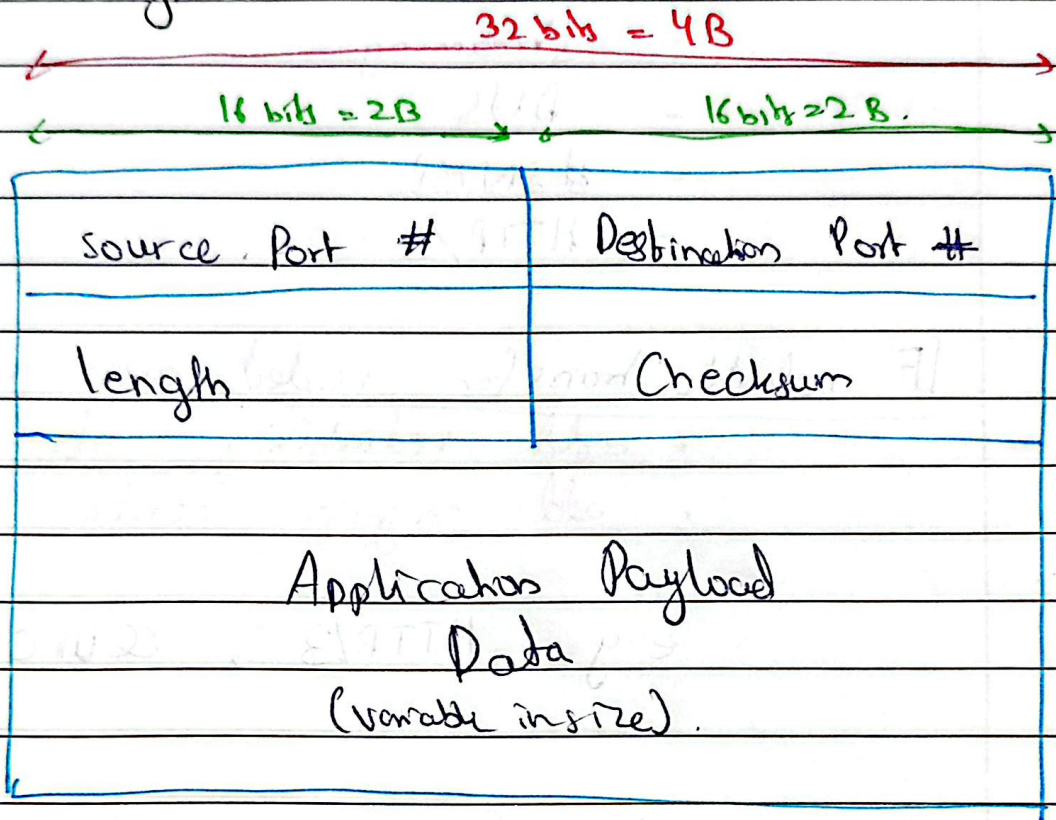
- Is passed as app layer message
- determines UDP header fields & values.
- creates UDP segment
- passes segment to IP (network layer)

receiver actions

- receives segment from IP (network layer).
- checks UDP checksum
- extracts APP-layer message
- demultiplexes message up to app ^{via} socket.

BHATTI

UDP segment header:



- Source Port # (16 bits) : identify sending app's port on source host
- Destination Port # (16 bits) : identify receiving app's port on destination host
- length (16 bits) : Specifies the length of UDP segment in **Bytes**. (**header + Data**).
Minimum value can be 8 Bytes (an empty packet with only header).

- **check sum (16 bits):** Provides error detection for segment

UDP Check sum:

This is used to check error (i.e. **flipped bits**) in transmitted segment.

How sender Calculates the Checksum:

- 1) **Treats** the entire UDP segment (header + payload) as a sequence of 16 bit integers.
- 2) **SUM** all 16-bit integers together using **1's complement arithmetic**
- 3) Takes the 1's complement. Flip all 0s to 1s & 1s to 0s of the final sum.
- 4) Places this result into **UDP checksum field**.

How receiver verifies the checksum.

1) Add up all 16 bits integers (including checksum itself) using 1's complement arithmetic.

2) Result check:

If all bits are one's (1111111111111111) , ~~NO~~ error.

If there is even one zero , ~~error~~

How do ~~the~~^{bits} all become one?

From sender.

Checksum = 1's complement of (Sum of Data words)

in receiver,

Total sum = Checksum + Sum of data words.

Total sum = 1's complement of (Sum of Data) +
sum of Data.

which should always lead to
~~0~~ 6(111111--) in no error.

Check sum example:

1) Consider we have 2 ~~16 bit~~ things
in UDP header:

1 with value = 58982 , 2 with value = 54613
~~data~~

now calculate check sum:

58982 =	①	1	1	0	①	0	1	1	0	①	0	1	1	0	①	0	1	1	0
54613 =	①	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
+																			
1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1																			

+

1 0 1 1 1 0 1 1 1 0 1 1 1 0 0

↓ inverted (1's complement)

check sum = 0 1 0 0 0 1 0 0 0 1 0 0 0 0 0 1

is remember
to
wrap
your
carry
& add
it
in 1's complement
addition

day/date

To find check sum:

- ① Standard binary addition
- ② Wrap around the carryout if generated & add it back.
- ③ Take 1's complement.

Checksum is weak protection:

• checksum is actually a poor way of protection.

because e.g. we are checking

$$\underline{5+4} = 9$$

header

checksum

but

$$6+3$$

$$3+6$$

$$4+5$$

$$2+7$$

all equal to 9.

so if we flip all header's ~~bits~~ some bits we might be able to pass checksum test